

EXPERIMENT-4

Student Details

Name: R.Varshitha

Roll Number: 24501a4450

Dataset: Position_Salaries.csv

1. Business Understanding

1.1 Problem Statement & Relevance

Salary often increases non-linearly as an employee moves to higher position levels. A straight-line model may not capture this curved relationship properly.

In this experiment, a Polynomial Regression model is developed to predict Salary using Level as the independent variable.

Problem Statement

Develop a machine learning model that predicts salary based on the employee's position level.

Relevance

- Helps estimate salary for a given job level.
- Supports salary benchmarking and compensation planning.
- Captures non-linear salary growth across position levels.
- Can help compare an offered salary with the learned salary trend.

1.2 Expected Benefits

- Better salary estimation for different position levels.
- Captures curved/non-linear relationships better than a simple straight line.
- Supports compensation analysis.
- Provides a visual salary trend using a polynomial regression curve.

2. Data Understanding

2.1 Dataset Structure

Dataset Name: **Position_Salaries.csv**

Variables used by the supplied program:

- Independent Variable: **Level**
- Target Variable: **Salary**

The exact number of records and the complete original dataset schema are not printed in the supplied source file, so they are not invented here.

2.2 Data Dictionary

Column	Description	Type	Role
Level	Position/job level	Numerical	Independent variable
Salary	Salary amount	Numerical	Target variable

2.3 Data Quality Observations

The program performs basic data inspection before modelling.

Source Code:

```
df.info()
df.describe()
df.head()
df.shape
print(df.isnull().sum())
print(df.duplicated().sum())
```

Observations

- Missing values are checked using `isnull().sum()`.
- Duplicate rows are checked and removed.
- A boxplot of Salary is used to inspect possible outliers.
- A scatter plot of Level versus Salary is used to observe the relationship.

```
df.drop_duplicates(inplace=True)
```

```
sns.boxplot(x=df['Salary'])
plt.show()
```

```
plt.scatter(x, y)
plt.xlabel("Level")
plt.ylabel("Salary")
plt.title("Level vs Salary")
plt.show()
```

3. Data Preparation

3.1 Data Cleaning

The following preprocessing steps are present in the supplied program:

- Load the Position Salaries dataset.
- Inspect dataset information and descriptive statistics.
- Check missing values.
- Check duplicate records.
- Remove duplicate records.
- Inspect Salary using a boxplot.

Source Code:

```
df = pd.read_csv('/content/Position_Salaries.csv')
print(df.isnull().sum())
print(df.duplicated().sum())
df.drop_duplicates(inplace=True)
```

3.2 Feature Engineering

Independent Variable (X)

Level

Source Code:

```
x = df['Level']
```

Dependent Variable (Y)

Salary

Source Code:

```
y = df['Salary']
```

Polynomial Transformation

The Level feature is transformed into polynomial terms up to degree 4.

```
poly = PolynomialFeatures(degree=4)
x_train_poly = poly.fit_transform(x_train.to_frame())
x_test_poly = poly.transform(x_test.to_frame())
```

3.3 Train-Test Split

Training Data = 80%

Testing Data = 20%

Random State = 42

Source Code:

```
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)
```

4. Modelling

4.1 Model Selection

Algorithm Used:

Polynomial Regression with Degree 4

Reason:

Polynomial Regression is suitable when the relationship between the independent and dependent variables is curved rather than strictly linear. Here, polynomial features are generated from Level and a LinearRegression model is trained on those transformed features.

General Regression Form:

$$\text{Salary} = b_0 + b_1(\text{Level}) + b_2(\text{Level}^2) + b_3(\text{Level}^3) + b_4(\text{Level}^4)$$

4.2 Model Training

Steps:

- Import required libraries.
- Load and inspect the dataset.
- Remove duplicate rows.
- Select Level as X and Salary as Y.
- Split the data into training and testing sets.
- Transform Level into fourth-degree polynomial features.
- Train the Linear Regression model on transformed training data.
- Generate predictions.

Source Code:

```
model = LinearRegression()
model.fit(x_train_poly, y_train)
print("Model Trained Successfully")

y_train_pred = model.predict(x_train_poly)
y_test_pred = model.predict(x_test_poly)
```

4.3 Model Outputs & Interpretation

The program compares actual and predicted test salaries and also predicts salary for a new Level value of 6.5.

Source Code:

```
result = pd.DataFrame({
    'Actual': y_test.values,
    'Predicted': y_test_pred
})
print(result)

new_level = pd.DataFrame([[6.5]], columns=['Level'])
new_level_poly = poly.transform(new_level)
predicted_salary = model.predict(new_level_poly)
print("Predicted Salary for Level 6.5 :", predicted_salary[0])
```

The exact printed prediction is not available from the source file alone because the dataset/output was not included.

5. Evaluation

5.1 Appropriate Metrics Selection

The following regression metrics are calculated for both training and testing data:

- Mean Absolute Error (MAE) - average absolute prediction error.
- Mean Squared Error (MSE) - average squared prediction error.
- Root Mean Squared Error (RMSE) - square root of MSE.
- R2 Score - indicates how well the model explains variation in Salary.

Source Code - Training:

```
print("MAE :", mean_absolute_error(y_train, y_train_pred))
print("MSE :", mean_squared_error(y_train, y_train_pred))
print("RMSE :", np.sqrt(mean_squared_error(y_train, y_train_pred)))
print("R2 Score :", r2_score(y_train, y_train_pred))
```

Source Code - Testing:

```
print("MAE :", mean_absolute_error(y_test, y_test_pred))
print("MSE :", mean_squared_error(y_test, y_test_pred))
print("RMSE :", np.sqrt(mean_squared_error(y_test, y_test_pred)))
print("R2 Score :", r2_score(y_test, y_test_pred))
```

5.2 Interpretation

- Lower MAE indicates smaller average prediction error.
- Lower MSE and RMSE indicate better prediction accuracy.
- An R2 Score closer to 1 indicates a stronger model fit.
- Training and testing metrics should be compared to understand how well the model generalizes.

The supplied Python file contains the metric calculations but not the executed numerical outputs, so exact metric values are not reported here.

6. Conclusion

The Position Salaries dataset is modelled using fourth-degree Polynomial Regression. The model learns the non-linear relationship between Level and Salary, generates training and testing predictions, evaluates performance using MAE, MSE, RMSE and R2 Score, and predicts a salary for Level 6.5.

Future improvements include:

- Compare different polynomial degrees.
- Use cross-validation for model selection.
- Compare Polynomial Regression with other regression algorithms.
- Use additional relevant features if a richer dataset is available.

7. Appendix

Polynomial Regression Curve

Source Code:

```
x_grid = np.arange(x.min(), x.max() + 0.1, 0.1)
x_grid = x_grid.reshape(-1, 1)
x_grid_df = pd.DataFrame(x_grid, columns=['Level'])

plt.figure(figsize=(6,4))
plt.scatter(x, y, color='red', label='Actual Data')
plt.plot(
    x_grid,
    model.predict(poly.transform(x_grid_df)),
```

```
    color='blue',  
    label='Polynomial Regression Curve'  
)  
plt.title("Polynomial Regression")  
plt.xlabel("Level")  
plt.ylabel("Salary")  
plt.legend()  
plt.show()
```

Illustrative layout of the plot (the exact executed output should be pasted from your Colab run):

